<Project Title>

Software Requirements Specification

Version <Version No.>
Status: <Version Status (draft, approved)>

Prepared by <Author's Name>
<Author's Position>

<XB Software Ltd.>

<Date of SRS creation>

Table of Contents

1. Introduction

1.1 Purpose
1.2 Document Conventions
1.3 Intended Audience and Reading Suggestions
1.4 Product Scope
1.5 References

2. Overall Description

2.1 Product Perspective
2.2 Product Functions
2.3 User Classes and Characteristics
2.4 Operating Environment
2.5 Design and Implementation Constraints
2.6 User Documentation
2.7 Assumptions and Dependencies

3. External Interface Requirements

3.1 User Interfaces
3.2 Hardware Interfaces
3.3 Software Interfaces
3.4 Communications Interfaces

4. Domain Model

5. System Features (Use Cases)

5.1 Use Case <Insert the specific use case name>
5.2 …

6. Other Nonfunctional Requirements

6.1 Performance Requirements
6.2 Safety Requirements
6.3 Security Requirements
6.4 Software Quality Attributes

7. Other Requirements

Revision History

Name	Date	Reason For Changes	Version	Date of Approval
< Author's Name>	<Date of Revision >	<Description of Made Changes>	<Version No.>	<Date of Approval >

1. Introduction

1.1 Purpose

Identify the product which software requirements are specified in this document, including the revision or release number. Describe the scope of the product that is covered by this SRS, particularly if this SRS describes only a part of the system or a single subsystem.

1.2 Document Conventions

For tracking the changes mark them in the document with color (changes, additions) or cross them (~~deleting~~).

1.3 Intended Audience and Reading Suggestions

Describe the different types of stakeholders that the document is intended for, such as developers, project managers, marketing staff, users, QA, and documentation writers. Describe what the rest of this SRS contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each stakeholder type.

EXAMPLE:

ID	Stakeholder	Description
S-1	Customer	Checking correspondence of business goals and functionality requirements to the expectations from implementing the product.
S-2	Development team	Forming the accurate vision of the project, detailed functional and nonfunctional requirements.
S-3	QA team	Making test-plans and test-cases.
S-4	PM/BA	Estimating the quote of the project, planning resources and the timeline of work.
S-5	User	Basing on this document the Terms of Service and the Privacy Policy are created.

1.4 Product Scope

Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies. If a separate Vision and Scope document is available, refer to it rather than duplicating its

contents here. If a SRS is planned to be composed step by step (the project consists of several iterations), it should include its own Vision and Scope document.

1.5 References

List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information, so that the reader could access a copy of each reference, including title, author, version number, date, and source or location (if applicable)

2. Overall Description

NOTE:
This section contains the general view of the project and the environment where it will be used, describes the expected user audience and constraints, assumptions and dependencies which can be identified.

2.1 Product Perspective

Describe the context and origin of the product being specified in this SRS. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product. If the SRS defines a component of a larger system, relate the requirements of the larger system to the functionality of this software and identify interfaces between the two.

Product Functions

Summarize the major functions the product must perform or must let the user perform. Details will be provided in Section 3.
A picture of the major groups of related requirements and their interactions, such as a top level data flow diagram or object class diagram, is often effective.

2.2 User Classes and Characteristics

Identify the various user classes that you anticipate will use this product. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes.

NOTE:
User classes are a subset of stakeholders, their description is included into the Vision and Scope document.

EXAMPLE:

ID	User classes	Description
U-1	Administrator	A signed up user who has completed the account activation. He owns expanded rights inside

		the portal. He performs the content pre moderation.
U-2	Signed up user	A user of the portal who has completed the sign up on the portal and the account activation.
U-3	Not signed up user	A user of the portal who has completed neither the sign up on the portal, nor the account activation. He owns limited rights inside the portal.

2.3 Operating Environment

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.
This information may be displayed at Vision and Scope document in details.

2.4 Design and Implementation Constraints

Describe any items or issues that will limit the options available to the developers. These might include:

- *specific technologies, tools, and databases to be used;*
- *hardware limitations (timing requirements, memory requirements);*
- *binding agreements or development standards;*
- *restrictions imposed by business rules;*
- *interfaces to other applications;*
- *standard data exchange format for example, XML;*
- *design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software);*
- *language requirements;*
- *communications protocols;*
- *security considerations.*

2.5 User Documentation

List the user documentation components that will be delivered along with the software. They may include user manuals, online help, and tutorials. Identify any known user documentation delivery formats or standards.

2.6 Assumptions and Dependencies

An assumption is something that you assume to be the case, even without proof.

List any assumed factors (as opposed to known facts) that could affect the requirements stated in the SRS. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the Vision and Scope document or the project plan).

NOTE:
In case the customer hasn't given any information on either of the paragraphs, write your vision in the section Assumptions or mark in the corresponding paragraph that the information wasn't given, so the development team will base on their thoughts and assumptions while developing the corresponding paragraph, taking into account and carefully following the interests of the client.

Dependencies - dependencies of the project on external factors like the date of the new OS version release.

If the system is planned to include the components developed in another project, then there is a dependency on its timely delivery.

3. External Interface Requirements

Requirements for the external interface define the tools, software or database elements with which the system or the component should interact.
Information from this section will give confidence that the system will properly interact with external components.
If different parts of the product have different external interfaces, then there should be a section with their description in the detailed requirements for each part.

3.1 User Interfaces

Describe the logical characteristics of each interface between the software product and the users. This may include sample screen images, any GUI standards or product family style guides that are to be followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. Define the software components for which a user interface is needed.

NOTE:

Document the user interface details such as dialogue window configuration in the user interface specification, not in SRS.
Details of the user interface design should be documented in a separate user interface specification.

Hardware Interfaces

Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.

3.2 Software Interfaces

Describe the connections between this product and other specific software components (name and version), including databases, operating systems, tools, libraries, and integrated commercial components.

Communications Interfaces

Describe the requirements associated with any communications functions required by this product, including e-mail, web browser, network server communications protocols, electronic forms, and so on. Define any pertinent message formatting. Identify any communication standards that will be used, such as FTP or HTTP. Specify any communication security or encryption issues, data transfer rates, and synchronization mechanisms.

4. Domain Model

<Optional section>

Domain model describes the entities structure composing the system, their attributes, operations and interconnection with the other entities.

5. System Features (Use Cases)

5.1 Use Case <Specify the title of a specific use scenario>

EXAMPLE:

Brief Description:	*<2 to 3 sentences describing the purpose and goals of use case>*
Business Trigger:	*< the business event that causes the first interaction>*
Preconditions:	*<the stable state(s) that the system must be in for the use case to start>*

Basic Flow: *<the complete flow of events that normally happens when everything goes right >*		
Assumptions: XXX *<the assumptions that are made for the basic flow, e.g. all items are for collection.>*		
Line	**System Actor Action**	**System Response**
1	XXX e.g. The sales assistant asks the system to process a new order. *<write a single sentence describing what the system actor does as an interaction across the system boundary>*	XXX e.g. The system asks the sales assistant for identification *<write a single sentence describing what the system does in response as an interaction across the system boundary, or, if absolutely essential, what the system does internally>*
2	XXX e.g. The sales assistant identifies themselves to the system *<add further lines as necessary until the flow is complete>*	XXX e.g. The system displays a blank order screen with a blank order line to the sales assistant *<keep the action and the response in pairs. If the system does more than one thing, then add another line for each additional action>*
3	Xxxx *<do not put conditions in the flow which refer to the alternate flow: describe in the alternate flow where it inserts itself>*	Xxxx *<when the flows are stable add hyperlink references to alternate and sub-flows>* (AFx)
Post Condition:	XXX *<the state of the system at the end of the basic flow, or things guaranteed to be true at the end of a successful use case>*	

Alternate Flow (AFx): XXXX *< the number and name of the alternate flow starting with an active verb>*		
If at line x in xxxx *<insert the name of the flow and the number of the line in which the condition occurs>* xxxx xxxx xxxx, then: *<define the condition under which the alternate flow is executed e.g. 'the item number entered is found to be invalid'. When the flows are stable, insert a hyperlink to the place where the flow inserts itself>*		
Line	**System Actor Action**	**System Response**
1	Xxxx *<write the lines just as you would in the basic course >*	Xxxx
2	Xxxx *<add further lines as necessary until the flow is complete>*	Xxxx
The use case terminates / The use case restarts at line x in x *<the last line must describe what happens next: the use case terminates; the use case restarts where it left off; the use case jumps back and restarts at an earlier step; the use case jumps forward and restarts at later step. When the flows are stable, insert a hyperlink to where the use case restarts>*		
Post Condition:	XXXX *<you might wish to describe post-conditions for an alternate flow where the use case terminates and the post-conditions are different from those of the basic flow>*	

Alternate Flow (AFx): XXXX *<add the names of further alternate flows as you think of them. Add the detail of the alternate flow after the basic flow has been detailed. It is possible to have extensions on the extensions. Write them exactly the same way as other extensions>*

Sub-Flow: XXX *<where there is procedure that is common to more than one flow in the use case, or where there are different flows following a case statement or selection statement, create sub-flows and 'call' them from the using flow. Name the sub-flows starting with an active verb>*

Line	System Actor Action	System Response
1	xxxx xxxx xxxx *<write the lines of the sub-flow just as you would in the basic course, describing what happens largely as interactions across the boundary >*	Xxxx
2	xxxx xxxx xxxx *<Add further lines as necessary>*	Xxxx *<When the flows are stable, insert a hyperlinks to where the flows restart>* Return1, Return 2

Business Rules:

1 xxxx *<include here a description of any business rules, detailed internal algorithms or procedures that are not part of the externally visible behaviour, but are vital to the functional definition of the system>*

2 xxxx *<if they have already been defined in a business rules document, then include the reference here instead of the description>*

Non Functional Requirements:

1 xxxx *<include here any non-functional requirements that are specific to this use case. Those which apply generally should go in the Non-Functional Requirements Document>*

2 xxxx *<include here any non-functional requirements that are specific to this use case. Those which apply generally should go in the Non-Functional Requirements Document>*

Data Requirements:

1 xxxx *<include here any specific data requirements. If these have already been specified in the Data Dictionary, then include here a reference to the entry instead>*

Activity Diagram:

<if there is complex iteration and selection, include an activity diagram, or a reference/hyperlink to one here. Activity diagrams should not duplicate or replace the text of the flows but augment it where prose is difficult to use to describe complex conditionality. There is no need to include every line in the use case as an activity.>

Prototype Screen:

<if you have a prototype screen for this use case, include it or a reference/hyperlink to it here. If might be at proof of concept or detailed level depending on the importance of the screen design to the user. Make sure the text of the use case is consistent with the prototype >

Screen Entry Exception Table: *<an optional table for simple exception handling of entries into fields on the prototype screen where adding an alternate flow would be overkill and the flow of the use case remains unchanged. This table could be used in place of the Data Dictionary>*

Field	Constraint	Response
1 e.g. Customer Surname	30 chars max	Message: “Exceeds 30 chars max – please re-enter”
2		

5.2 …

6. Other Nonfunctional Requirements

6.1 Performance Requirements

State performance requirements

6.2 Safety Requirements

Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Refer to any external policies or regulations that state safety issues that affect the product’s design or use.

6.3 Security Requirements

Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication requirements. Refer to any external policies or regulations containing security issues that affect the product.

6.4 Software Quality Attributes

Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.

7. Other Requirements

Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project

Appendix A: Glossary

EXAMPLE:

Term	Description
Project	Modification of the existent Reputation.ca website
SERP	Search Engine Result Page. Page with the results available to the User after search (clicking on Get Quote button Free Analysis). This page shall contain 1-20 results.
TDB	To be detailed
Reputation Management	Management of results which were obtained on SERP (view, edit, delete, pay). Available only for the Authorized User.
AJAX	A group of interrelated Web development techniques used on the client-side to create asynchronous Web applications (see http://en.wikipedia.org/wiki/Ajax_(programming))

Appendix B: Analysis Models

An additional optional section which includes models referring to the product described in the SRS, e.g. diagrams of data flows, class diagrams, state machine diagrams, entity-relationship models.

Appendix C: Interface Examples

EXAMPLE1:

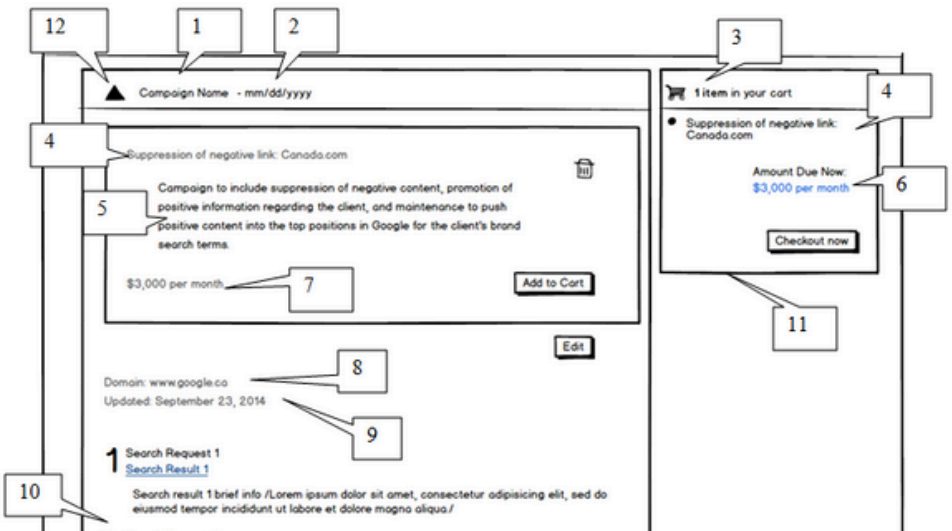
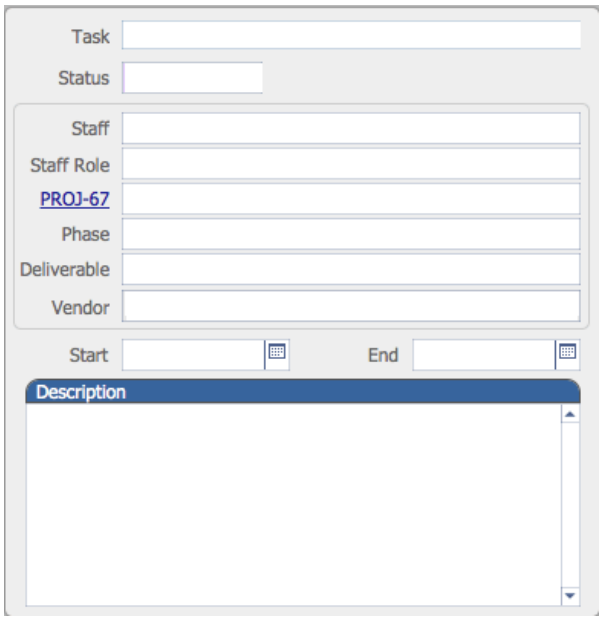


Figure 4. Campaigns

Campaign Name (1) – The keywords entered for search.
Date (2) – The date of the first search of the keywords (1)
Search Result (10) – The results of the latest search for the specified key words (1)
Update date (9) – The date of the last search for the specified key words (1)
Due Amount (6) – The sum payable for all selected items from all Campaigns.
Cart (11) – For all items selected from all Campaigns.
Arrow button (12) – By clicking on this button, the opened Campaign will be folded up. If there are several Campaigns, they shall be displayed as an accordion, each may be unrolled.

EXAMPLE2:



Field	Description
Task Name	Editable text field. Max. number of symbols: 100 Symbols type: letters and digits only, both capital and lowercase
Status	Not editable text field. Auto filled.
Staff	Not editable text field. Auto filled.
Staff Role	Not editable text field - Link. Auto filled.
PROJ-67 (Project Number)	Not editable dynamic field label from JSON. Auto filled. onClick handler shall be put on it.
Project Name	Not editable text field. Auto filled.
Phase	Not editable text field. Auto filled.
Deliverable Type	Not editable text field. Auto filled.
Vendor	Not editable text field. Auto filled.
Start Date	Editable. Field mask: mm/dd/yyyy Auto-changeable after the task position change.
End Date	Editable. Field mask: mm/dd/yyyy Auto-changeable after the task position change.
Description	Editable text field. Max. number of symbols: 10,000 Symbols type: letters (capital and lowercase), digits and punctuation marks.

Appendix D: To Be Determined List

This section should include all questions and issues which need clarification.